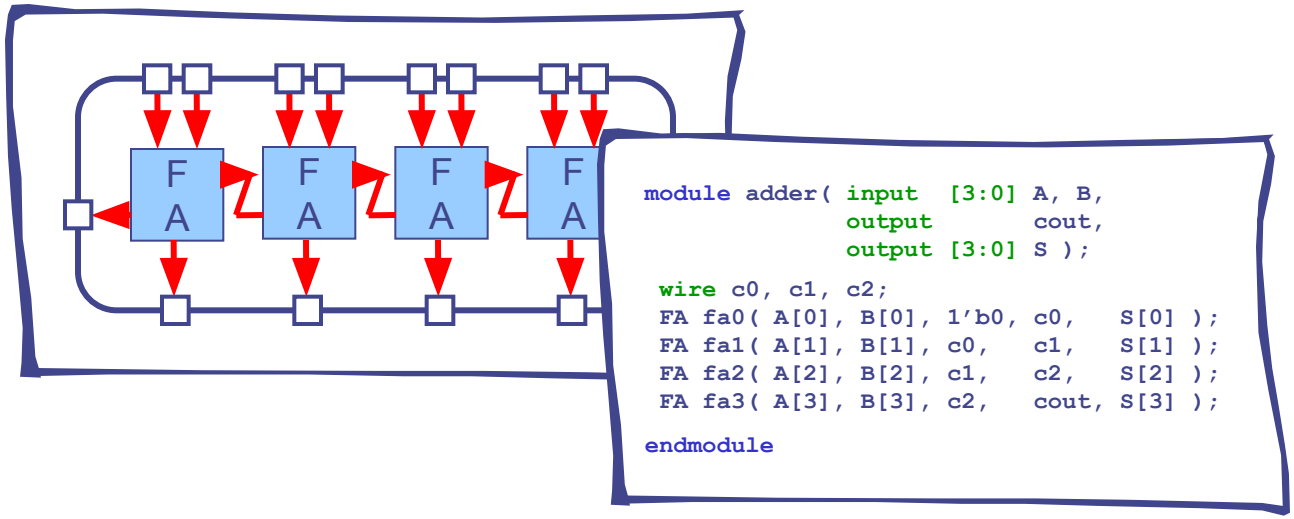
Updated January 2023

Synthesizable System Verilog 1 - Fundamentals



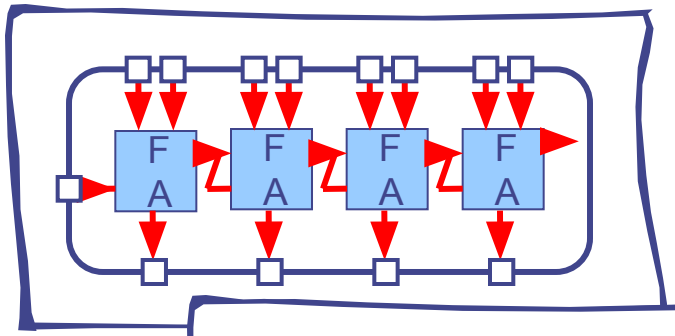
Prof. Michael Taylor

What is Synthesizeable SystemVerilog (SV)?

- In this class and in the real world, SystemVerilog is a specification language, not a programming language.
  - Draw your schematic and state machines and then *transcribe* it into SV.
  - When you sit down to write SV you should know *exactly* what you are implementing.
- We are constraining you to a subset of the language for two reasons
  - These are the parts that people use to design real processors
  - Steer you clear of problematic constructs that lead to bad design.

SV Fundamentals

- What is System Verilog?
- Data types
- Structural SV
- RTL SV
  - Combinational Logic
  - Sequential Logic




```
module adder( input  [3:0] A, B,
              output       cout,
              output [3:0] S );

 wire c0, c1, c2;
 FA fa0( A[0], B[0], 1'b0, c0,   S[0] );
 FA fa1( A[1], B[1], c0,   c1,   S[1] );
 FA fa2( A[2], B[2], c1,   c2,   S[2] );
 FA fa3( A[3], B[3], c2,   cout, S[3] );

endmodule
```

Bit-vectors are the primary data type in Synthesizable SV

A bit can take on one of four values

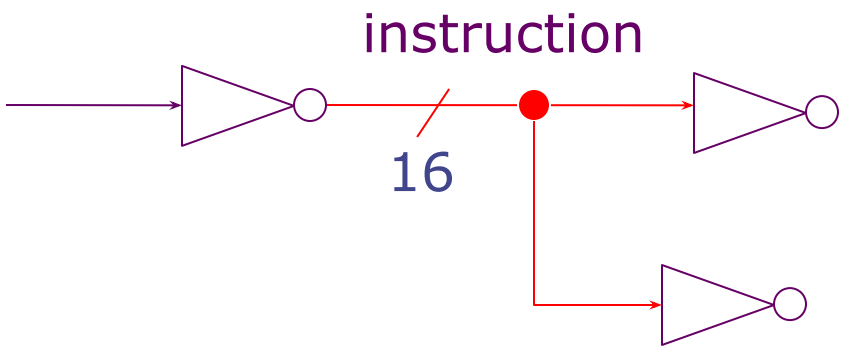
Value	Meaning
0	Logic zero
1	Logic one
X	Unknown logic value
Z	High impedance, floating

In the simulation waveform viewer, Unknown signals are RED. There should be no red after reset.

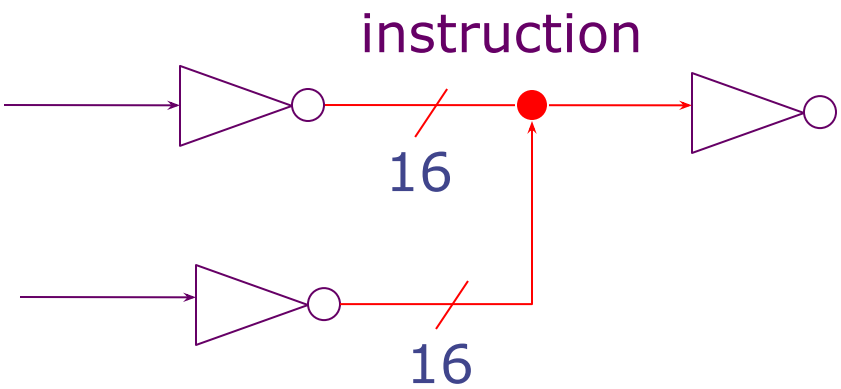
An X bit might be a 0, 1, Z, or in transition. We can set bits to be X in situations where we don't care what the value is. This can help catch bugs and improve synthesis quality.

"logic" is used to denote a hardware net that has a single driver but possibly multiple outputs

```
logic [15:0] instruction;
```



Legal

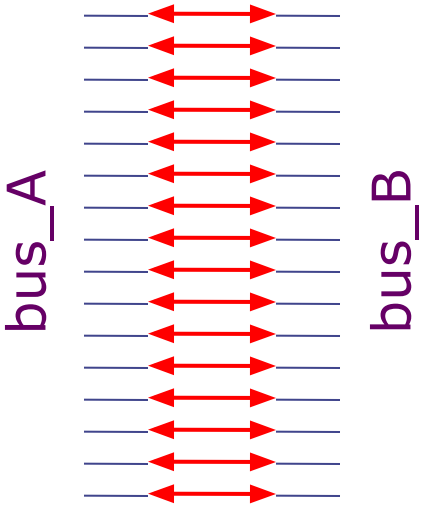


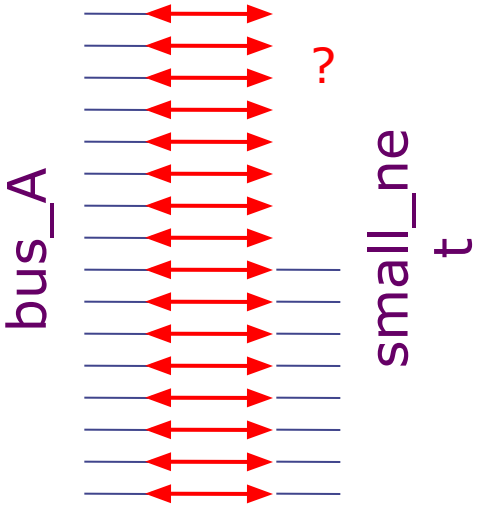
Illegal

`wire` is used to denote a hardware net that has >=1 drivers, or that has unknown directionality

```
wire [15:0] bus_A;
wire [15:0] bus_B;
wire [ 7:0] small_net;
```

Absolutely no type safety when connecting nets!





2012: You will rarely use the wire keyword in this class.

Bit literals

4'b10_11

Underscores are ignored

Base format (d,b,o,h)

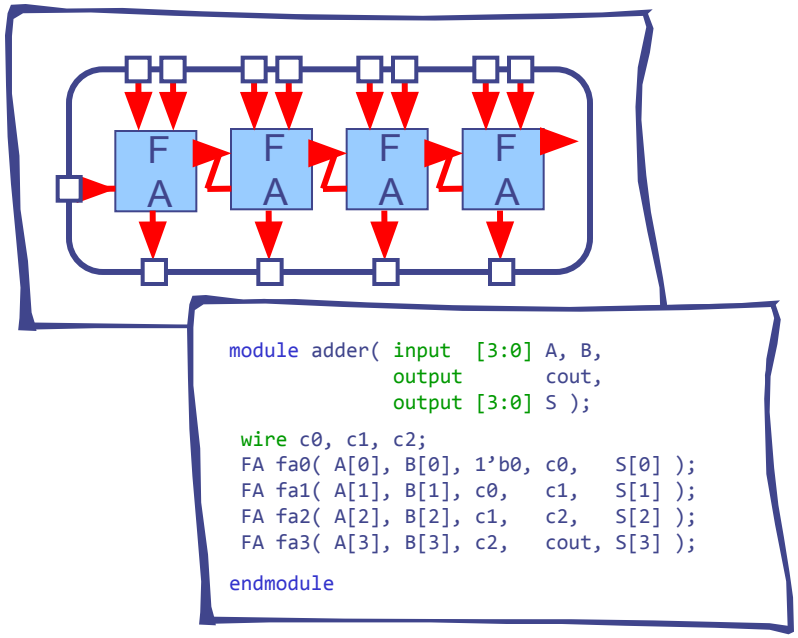
Decimal number representing size in bits

We'll learn how to actually assign literals to nets a little later

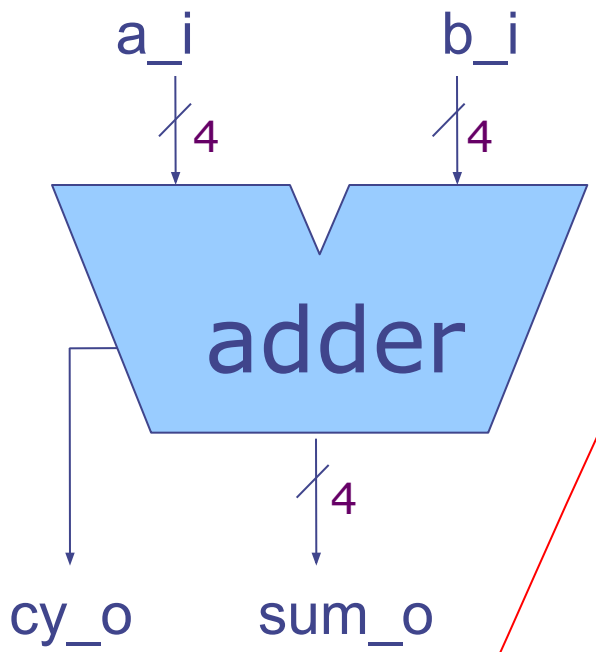
- Binary literals
  - **8'b0000_0000**
  - **8'b0xx0_1xx1**
- Hexadecimal literals
  - **32'h0a34_def1**
  - **16'haxxx**
- Decimal literals
  - **32'd42**

SV Fundamentals

- History of hardware design languages
- Data types
- Structural SV
- RTL SV



A SV module has a name and a port list

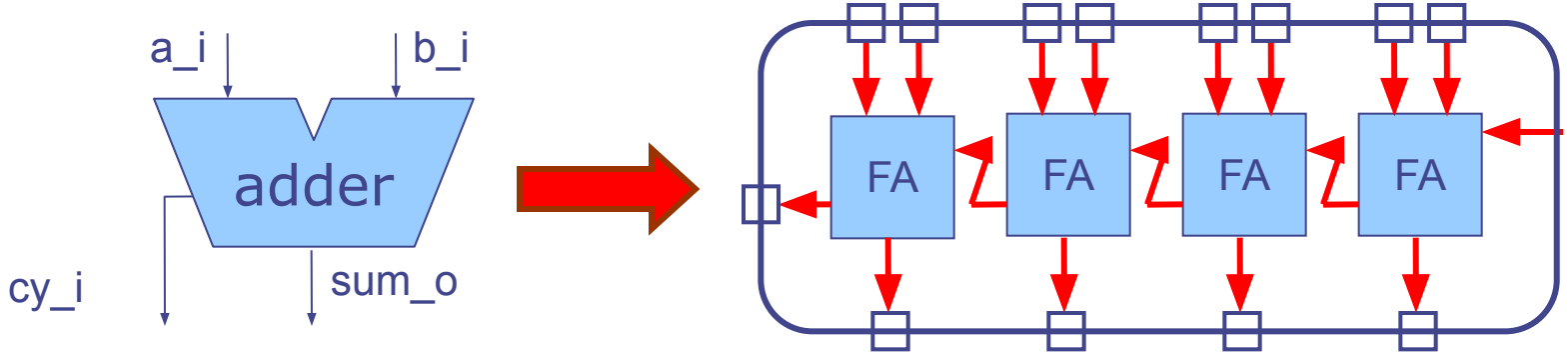



```
module adder(input  [3:0] a_i
             ,input  [3:0] b_i
             ,output       cy_o
             ,output [3:0] sum_o );

  // HDL modeling of
  // adder functionality
endmodule
```

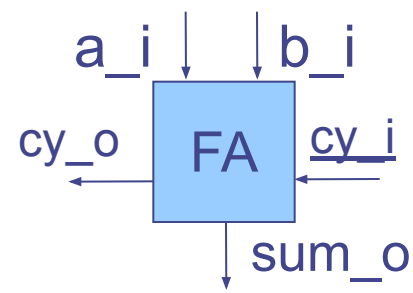
Ports must have a direction and a bitwidth. In this class we use `_i` to denote in port variables and `_o` to denote out port variables.

Note the semicolon at the end of the port list!

A module can instantiate other modules

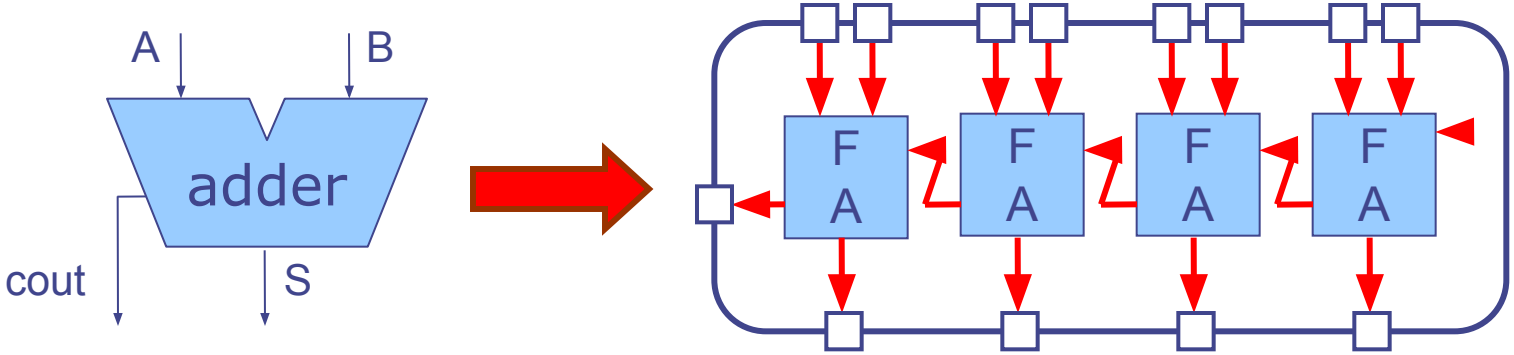



```
module adder( input  [3:0] a_i, b_i
             ,output       cy_o
             ,output [3:0] sum_o );

  logic c0, c1, c2;
  FA fa0( ... );
  FA fa1( ... );
  FA fa2( ... );
  FA fa3( ... );

endmodule
```




```
module FA( input a_i, b_i, cy_i
          ,output cy_o, sum_o);
  // HDL modelling of 1 bit
  // full adder functionality
endmodule
```

Connecting modules




```
module adder( input  [3:0] a_i, b_i
             ,output       cy_o
             ,output [3:0] sum_o );

 logic c0, c1, c2;
 FA fa0( a_i[0], b_i[0], 1'b0, c0,    sum_o[0] );
 FA fa1( a_i[1], b_i[1], c0,   c1,    sum_o[1] );
 FA fa2( a_i[2], b_i[2], c1,   c2,    sum_o[2] );
 FA fa3( a_i[3], b_i[3], c2,   cy_o,  sum_o[3] );

endmodule
```

Carry Chain

Only connect ports by name and not by position.

Connecting ports by ordered list is compact but bug prone:

```
FA fa0( a_i[0], b_i[0], 1'b0, c0, sum_o[0] );
```

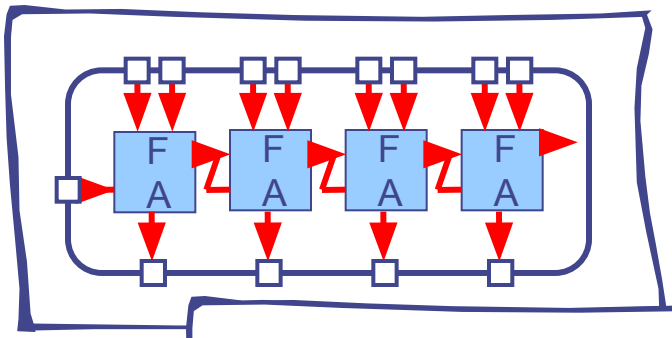
Connecting by name is less compact but leads to fewer bugs. This is how you should do it in this class. You should also line up like parameters so it is easy to check correctness.

```
FA fa0( .a_i(a_i[0])
        ,.b_i(b_i[0])
        ,.cy_i(1'b0)
        ,.cy_o(c0)
        ,.sum_o(sum_o[0])
       );
```

Connecting ports by name yields clearer and less buggy code. In the slides, we may do it by position for space. But you should do it by name and not position.

Verilog Fundamentals

- History of hardware design languages
- Data types
- Structural Verilog
- RTL
  - Combinational
  - Sequential




```
module adder( input  [3:0] A, B,
              output       cout,
              output [3:0] S );

 wire c0, c1, c2;
 FA fa0( A[0], B[0], 1'b0, c0,   S[0] );
 FA fa1( A[1], B[1], c0,   c1,   S[1] );
 FA fa2( A[2], B[2], c1,   c2,   S[2] );
 FA fa3( A[3], B[3], c2,   cout, S[3] );

endmodule
```

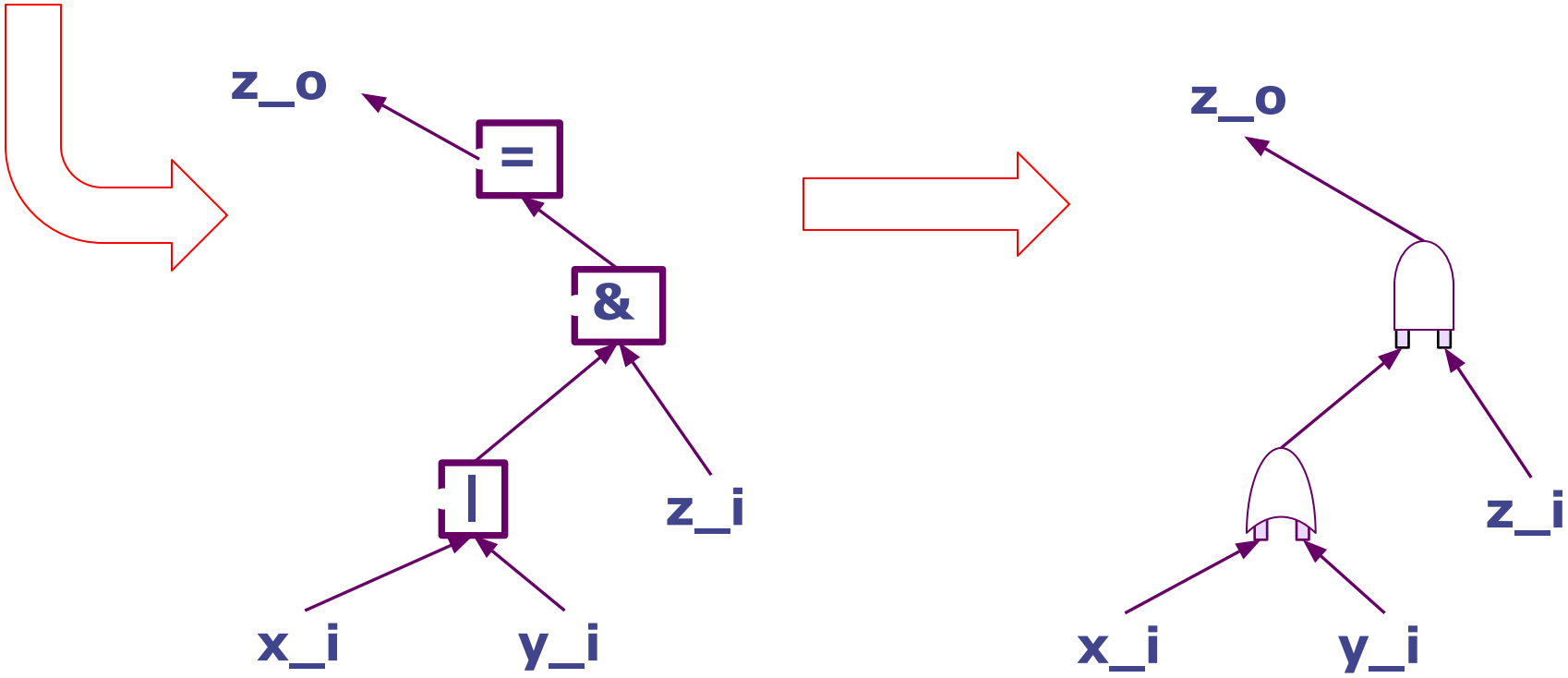
Combinational Verilog: `assign`

very straightforward mapping to hardware

variables are names of wires; operators are gates

Language-defined operators

```
assign z_o = (x_i | y_i) & z_i;
```



parse tree

tree of gates

A module’s behavior can be described in many different ways but it should not matter from outside

Example: mux4

mux4: Using continuous assignments to generate combinational logic

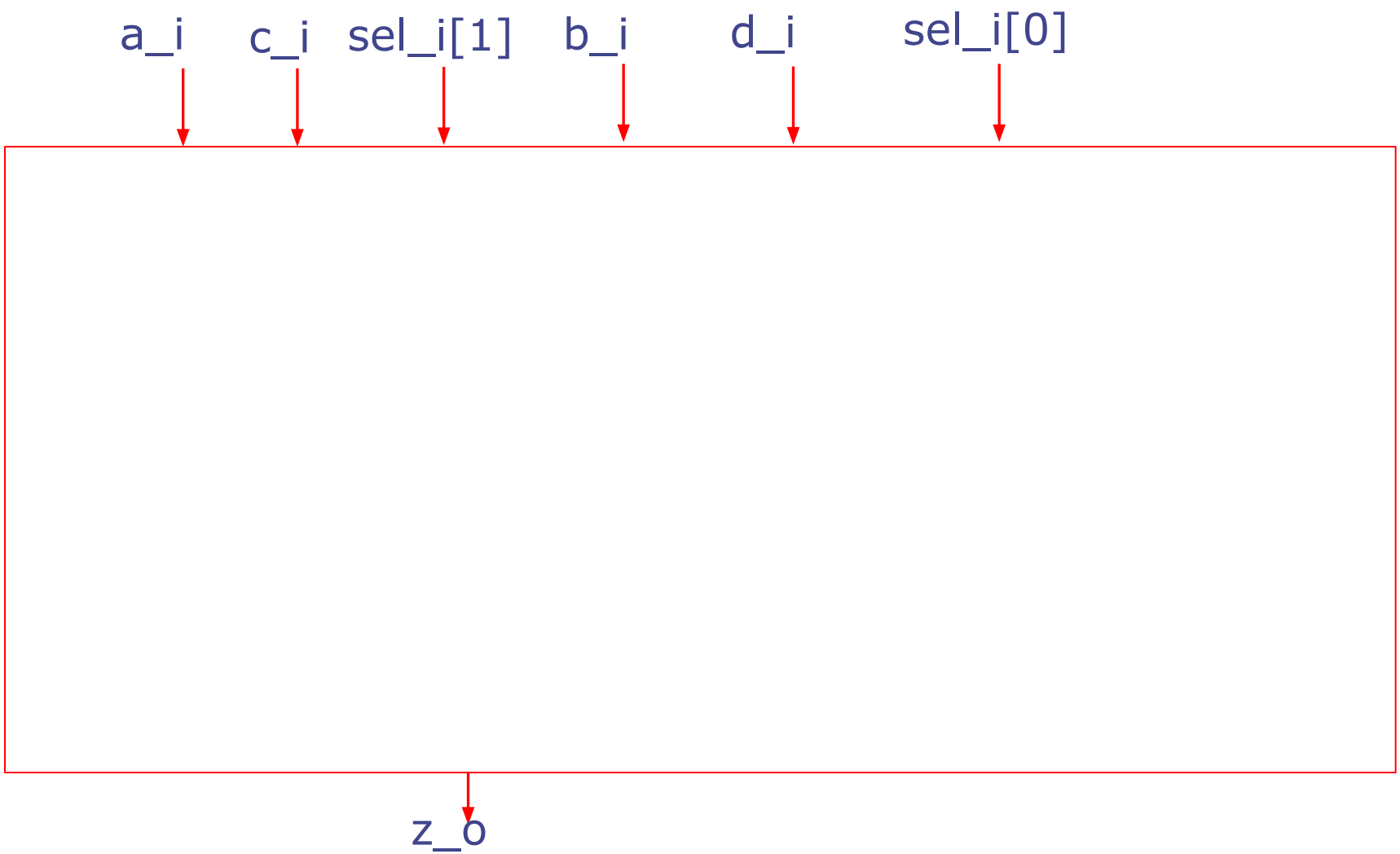
```
module mux4( input  a_i, b_i, c_i, d_i
            ,input [1:0] sel_i
            ,output z_o );

logic t0, t1;

assign z_o = ~((t0 | sel_i[0]) & (t1 | ~sel_i[0]));
assign t1  = ~((sel_i[1] & d_i) | (~sel_i[1] & b_i));
assign t0  = ~((sel_i[1] & c_i) | (~sel_i[1] & a_i));

endmodule
```

A couple of combinational trees that connect to each other

The order of these continuous **assign** statements in the source code does not affect functionality – they are just specifying a bunch of gates – *a combinational cloud*.

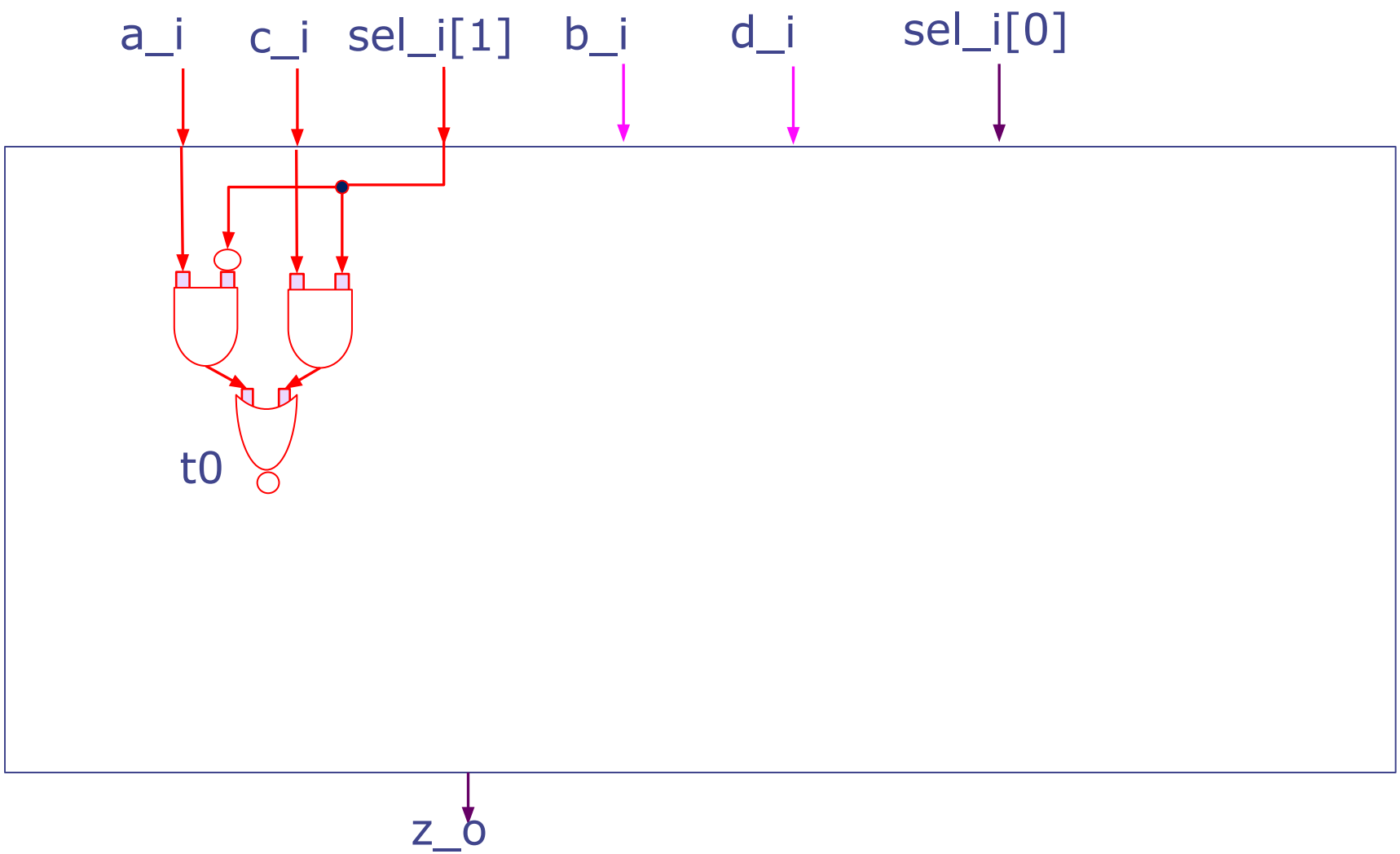
Any time an input to the combinational cloud changes, it propagates through the cloud of gates and the outputs are updated. (Be careful not to create combinational cycles!)

```
module mux4( input  a_i, b_i, c_i, d_i
            ,input [1:0] sel_i
            ,output z_o );
```



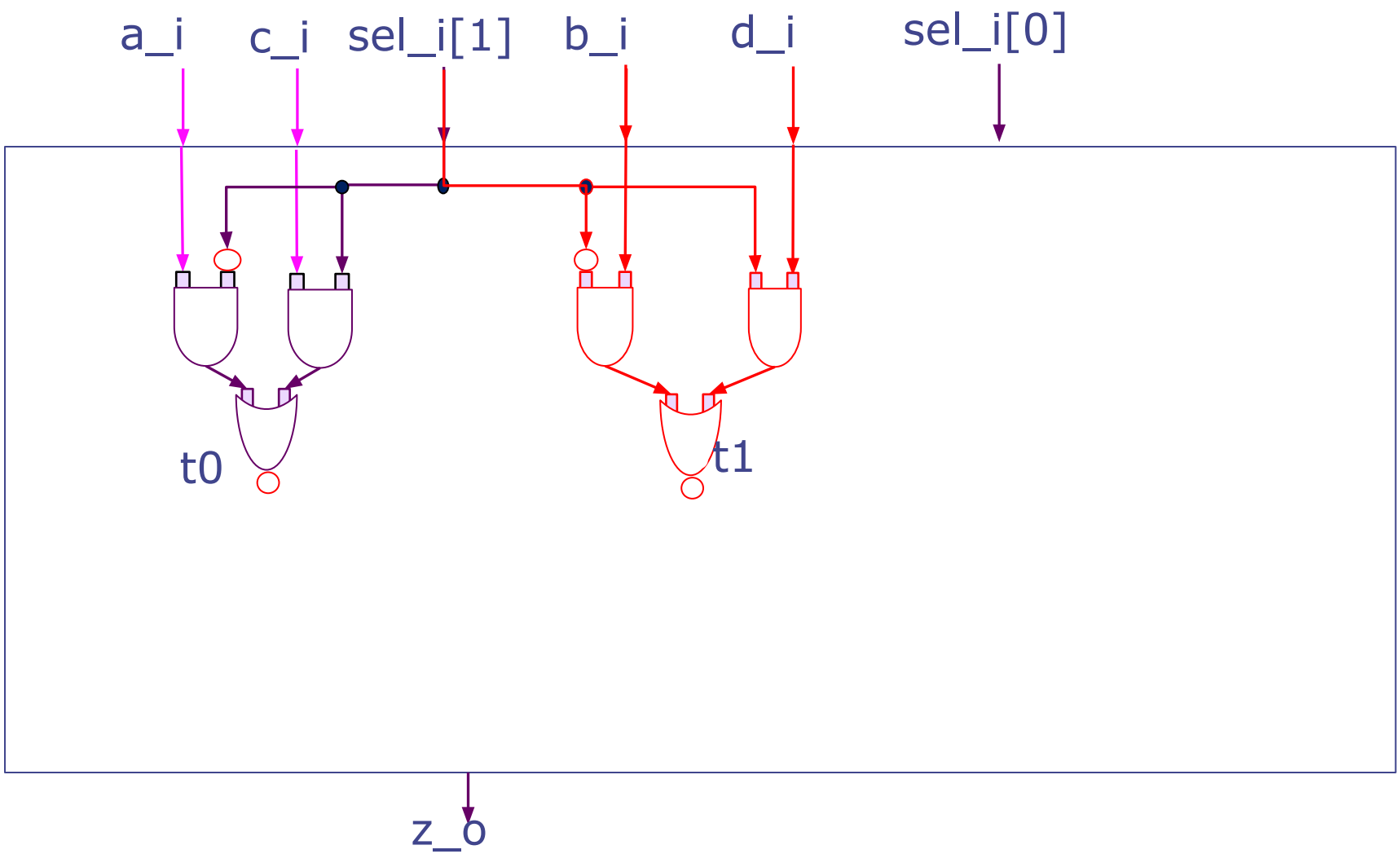
Create the module inputs and outputs

```
assign t0  = ~((sel_i[1] & c_i) | (~sel_i[1] & a_i));
```



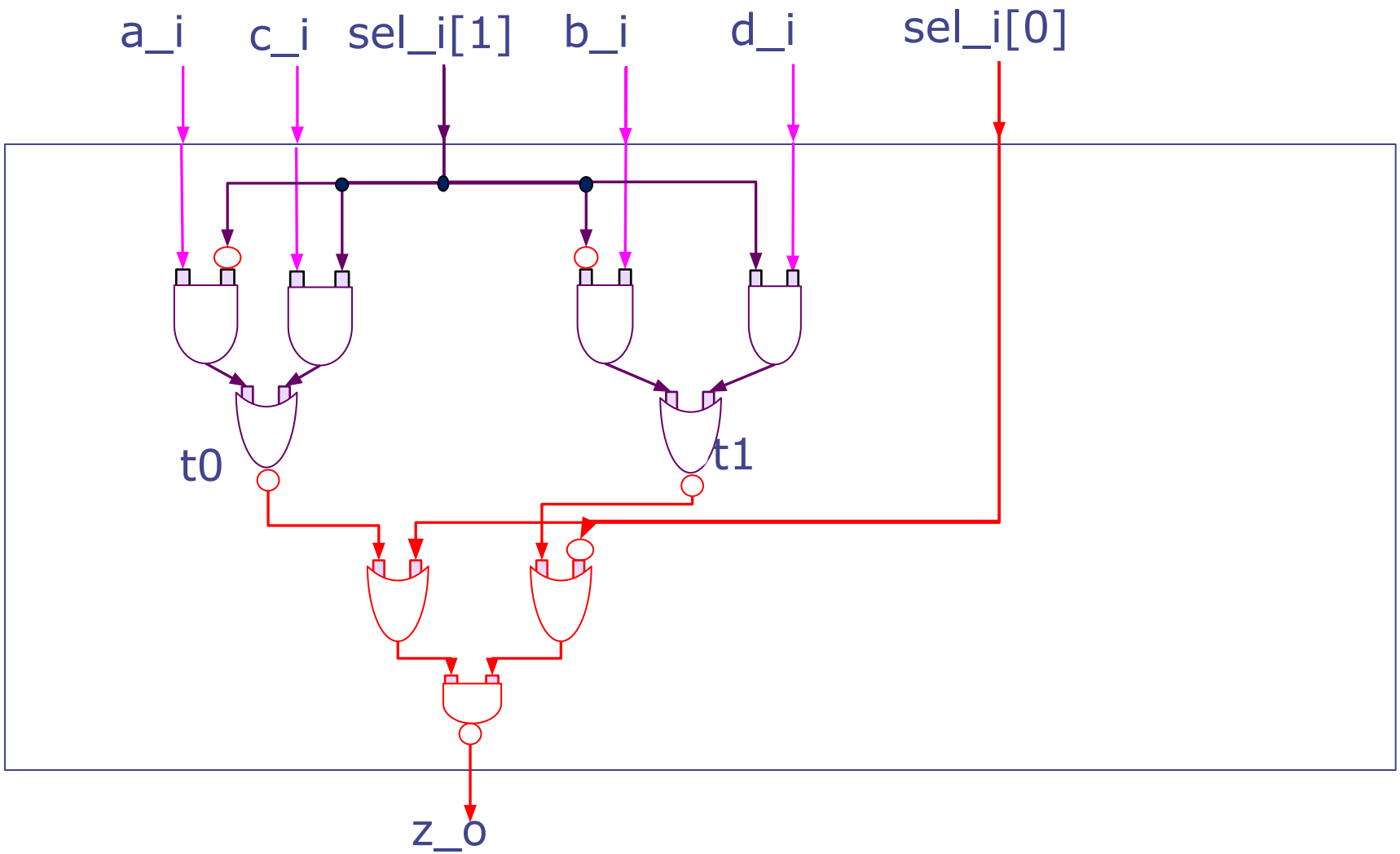
Generate the logic tree for t0

```
assign t1  = ~((sel_i[1] & d_i) | (~sel_i[1] & b_i));
```



Generate the logic tree for t1

```
assign z_o = ~((t0 | sel_i[0]) & (t1 | ~sel_i[0]));
```



Generate the logic tree for z_o

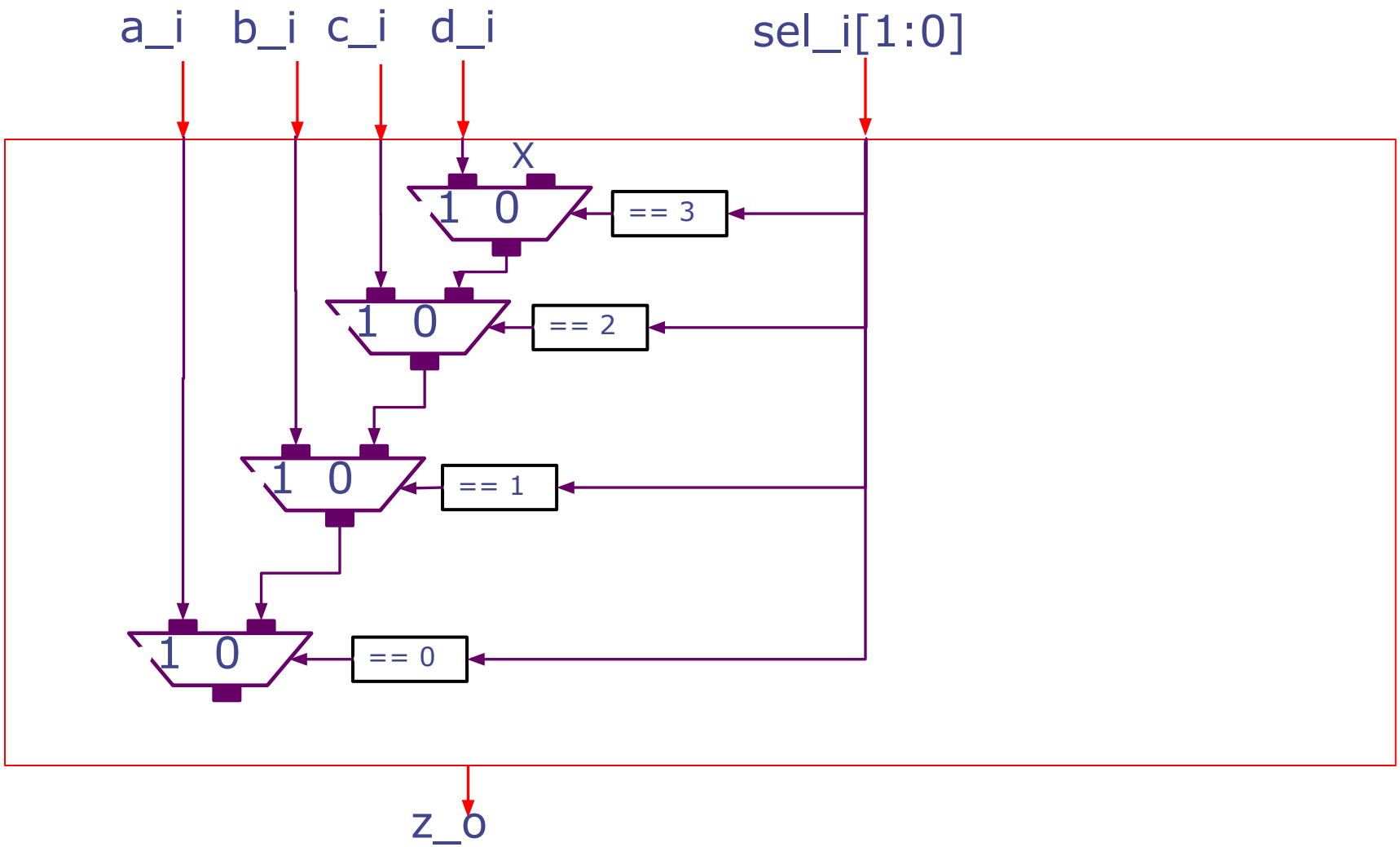
mux4: Using ? :

```
// Four input multiplexer
module mux4( input  a_i, b_i, c_i, d_i,
             input [1:0] sel_i,
             output z_o);

  assign z_o = ( sel_i == 0 ) ? a_i :
               ( sel_i == 1 ) ? b_i :
               ( sel_i == 2 ) ? c_i :
               ( sel_i == 3 ) ? d_i : 1'bx;

endmodule
```

*If **sel_i** is X or Z, without the 1'bX condition, it would get d_ in RTL simulation but maybe not in gate-level. Bad!*

Having the 1'bx will help make sure your (post-synthesis) gate-level simulation looks the same as your RTL-level sim.

```
module mux4( input  a_i, b_i, c_i, d_i
            ,input [1:0] sel_i
            ,output z_o );
```



Create the module inputs and outputs

mux4: Using combinational "always_comb" or "always @(*)" block

```
module mux4( input  a_i, b_i, c_i, d_i
            ,input [1:0] sel_i
            ,output logic z_o );

  logic t0, t1;

  always_comb     // system verilog; replaces always @(*)
  begin
    t0  =   (sel_i[1] & c_i) | (~sel_i[1] & a_i);
    t1  = ~((sel_i[1] & d_i) | (~sel_i[1] & b_i));
    t0  = ~t0;
    z_o = ~( (t0 | sel_i[0]) & (t1 | ~sel_i[0]) );
  end

endmodule
```

Within the always_comb block, the synthesis tool synthesizes the lines in order. Each L-value (variable to the left of =) creates a name for the wire that is at the top of a logic tree. If a variable is assigned again (like t0), then the mapping is updated – no cycles are created.

“always_comb” permit more advanced combinational idioms like if/else and case

```
module mux4( input  a_i,b_i,c_i,d_i
            ,input [1:0] sel_i
            ,output logic z_o);
```

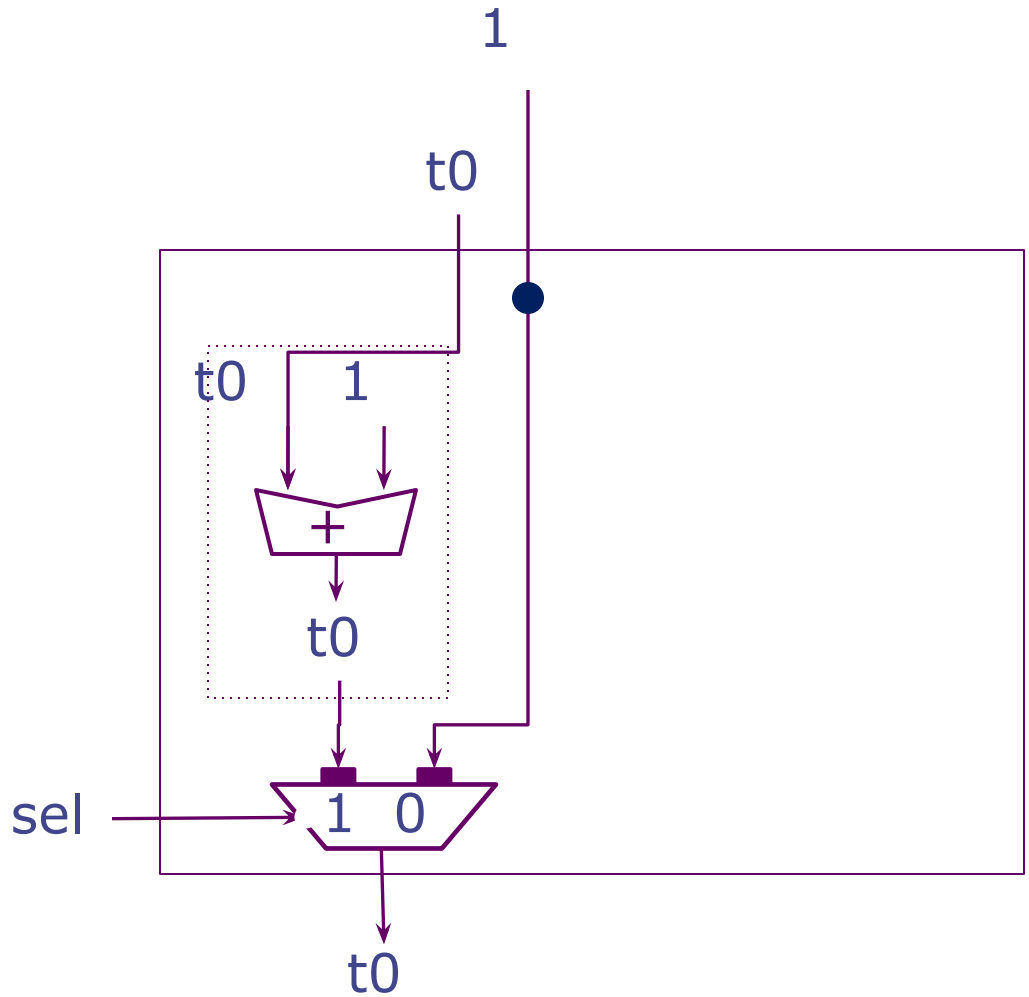
Do this to avoid RTL versus gate-level sim mismatches. If none of these match, behavioral will just use last value. Gate level will give you an X or random.

```
  always_comb
    begin
      if (sel_i == 2'd0 )
        z_o = a_i;
      else if (sel_i == 2'd1)
        z_o = b_i;
      else if (sel_i == 2'd2)
        z_o = c_i;
      else if (sel_i == 2'd3)
        z_o = d_i;
      else
        z_o = 1'bx;
    end
  endmodule
```

```
always_comb
   begin
    case ( sel_i )
      2'd0 : z_o = a_i;
      2'd1 : z_o = b_i;
      2'd2 : z_o = c_i;
      2'd3 : z_o = d_i;
      default: z_o = 1'bX;
     endcase
  end
endmodule
```

Synthesis of if

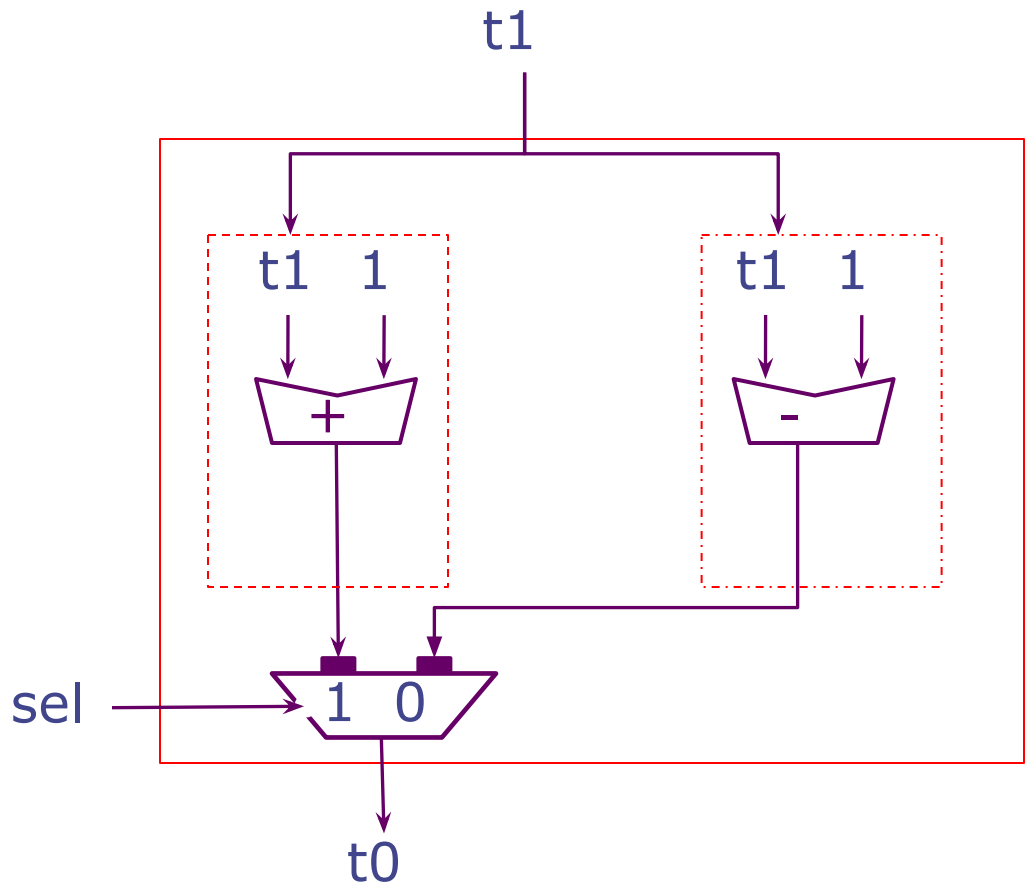
```
logic t0, t1, sel;

always_comb
 begin
  t0 = 1'b1;

  if (sel)
   t0 = t0+1;

 …
 end
```



Synthesis of single-output if/else

```
logic t0, t1, sel;

always_comb
 begin
 …

   if (sel)
    t0 = t1+1;
   else
    t0 = t1-1;

 …
 end
```

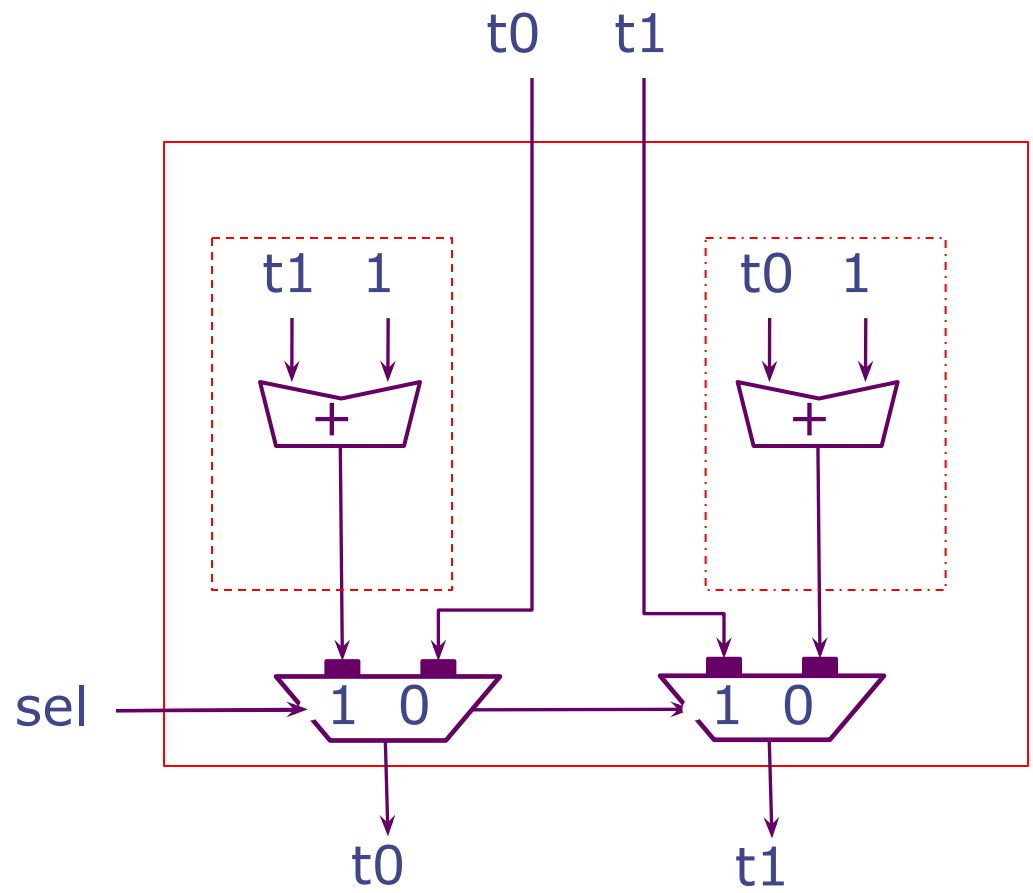
L-values



Synthesis of multi-output if/else

```
logic t0, t1, sel;

always_comb
 begin
 …

   if (sel)
    t0 = t1+1;
   else
    t1 = t0+1;

 …
 end
```

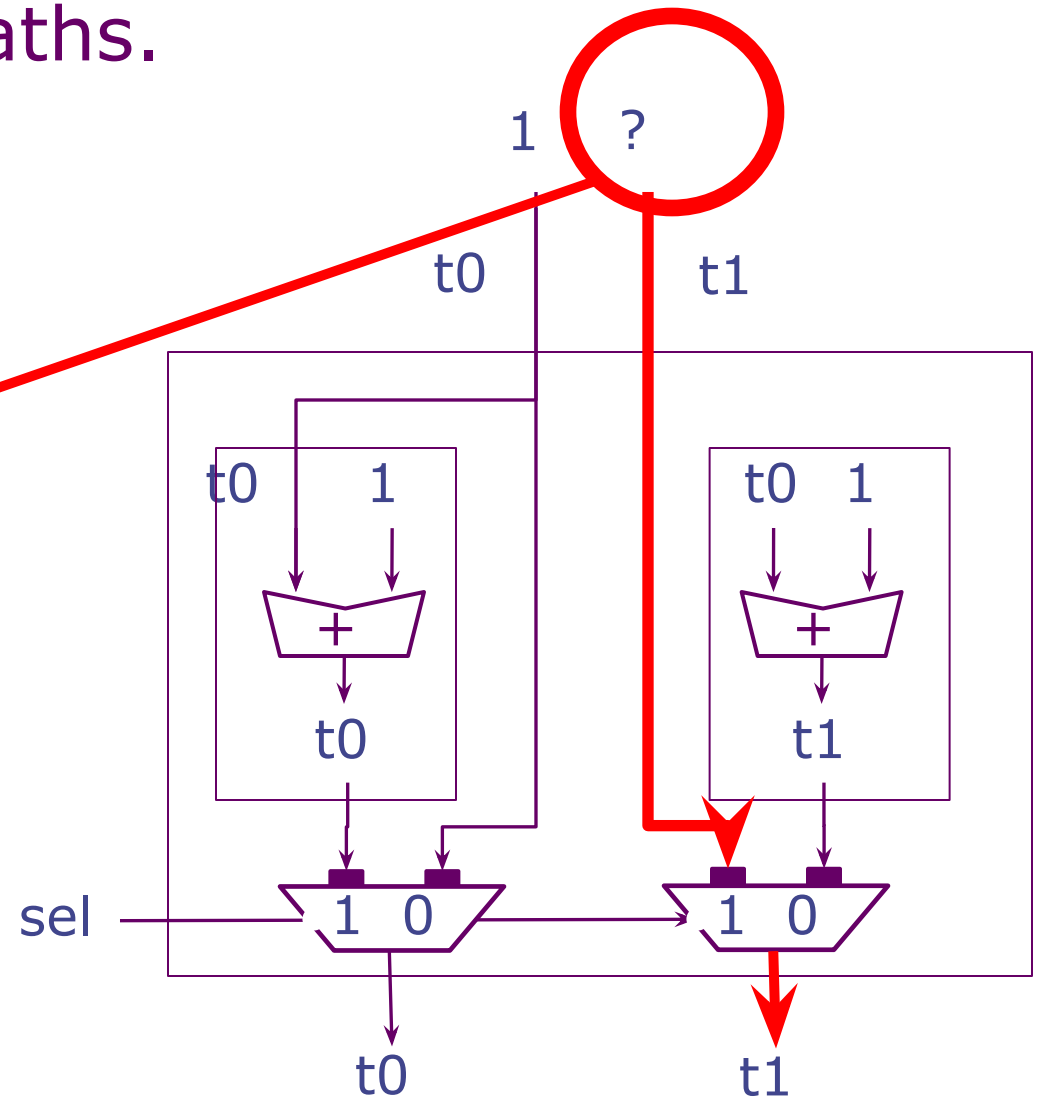
L-values



Note: a mux is created for every L-value written by all branches of the if/else or case statement.

Synthesis of if/else: beware of not setting all outputs on all paths.

```
logic t0, t1, sel;

always_comb
 begin
  t0 = 1'b1;

   if (sel)
  t0 = t0+1;
   else
  t1 = t0+1;

  …
  end
```



Note: no L-value should be undefined on any path; behavior is undefined; Verilog will create a latch (ugh)!

Make sure your case statements are complete

To avoid mismatches in behavior between RTL and gate-level sim

```
module mux3( input  a_i, b_i, c_i
            ,input [1:0] sel_i
            ,output logic z_o );

always_comb @( * )

  begin
    case ( sel_i )
      2'd0 : z_o = a_i;
      2'd1 : z_o = b_i;
      2'd2 : z_o = c_i;
      default : z_o =  <value or 2'bX>
    endcase
  end

endmodule
```

OR

```
module mux3( input  a_i, b_i, c_i
            ,input [1:0] sel_i
            ,output logic z_o );

always_comb @( * )

  z_o = <value or 2'bX>

  begin
    case ( sel_i )
      2'd0 : z_o = a_i;
      2'd1 : z_o = b_i;
      2'd2 : z_o = c_i;
    endcase
  end

endmodule
```

Parameterized mux4

default value

```
module mux4 #( parameter width_p = 1 )
             ( input[width_p-1:0]  a_i, b_i, c_i, d_i
              ,input [1:0] sel_i
              ,output[width_p-1:0] z_o );

  wire [width_p-1:0] t0, t1;

  assign t0  = (sel_i[1]? c_i : a_i);
  assign t1  = (sel_i[1]? d_i : b_i);
  assign z_o = (sel_i[0]? t0: t1);
endmodule
```

Parameterization is a good practice for reusable modules

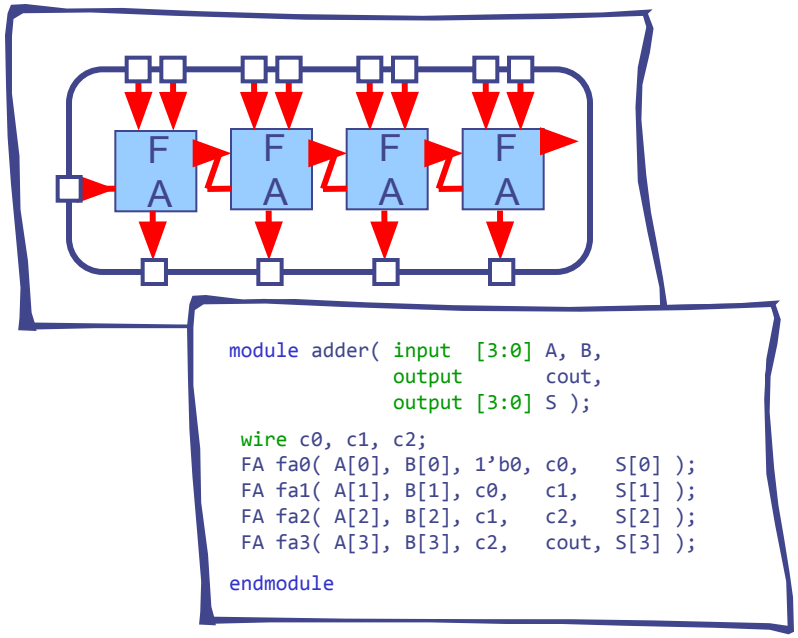
Writing a mux*n* is challenging, but can be done with "idiomatic" verilog.

Instantiation

```
mux4#(.width_p(32))
alu_mux
(.a_i (op1),
 .b_i (op2),
 .c_i (op3),
 .d_i (op4),

 .sel_i(alu_mux_sel),
 .z_o(alu_mux_out)
);
```

Verilog Fundamentals

- History of hardware design languages
- Data types
- Structural Verilog
- RTL
  - Combinational
  - Sequential



Sequential Logic: Creating a flip flop

```
logic q_r, q_n;
always_ff @( posedge clk )
   q_r <= q_n;
```

1) This line simply creates two wires, one called `q_r` and the other called `q_n`.

2) `always_ff` keyword indicates our intent to create **f**lip-**f**lops; you could use the `always` keyword instead, but this makes it clear what you want!

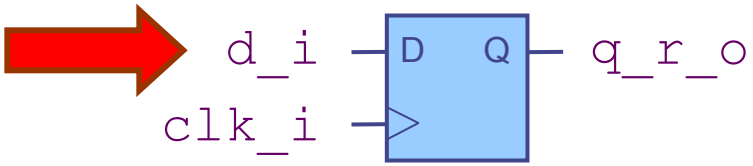
3) `@( posedge clk )` indicates that we want these registers to be triggered on the positive edge of the `clk` clock signal. You could also say `negedge`.

4) Combined with 2) and 3), the `<=` creates a register whose input is wired to `q_n` and whose output is wired to `q_r`. Use `_r` to indicate a wire that comes directly out of a register, and `_n` (i.e., next) to indicate a wire that goes directly into one, and becomes the new output on the next cycle.

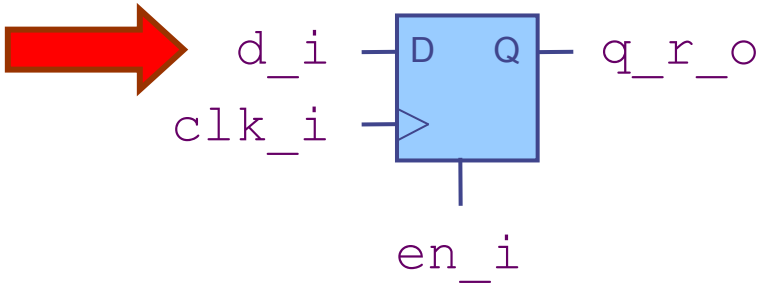
note: always use <= with `always_ff` and = with `always_comb`

Sequential Logic: flip-flop idioms

```
module FF0 (input clk_i, input  d_i
           ,output logic q_r_o);
always_ff @( posedge clk_i )
  begin
     q_r_o <= d_i;
  end
endmodule
```



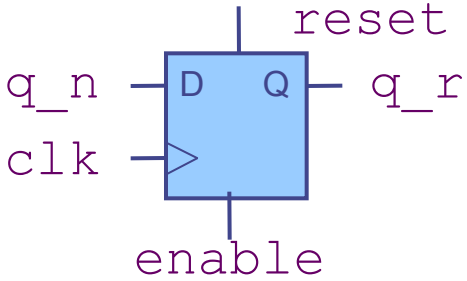

```
module FF (input clk, input  d_i
          ,input  en_i, output logic q_r_o);
always_ff @( posedge clk )
  begin
    if ( en_i )
     q_r_o <= d_i;
  end
endmodule
```



idiom: flip-flops with reset

```
always_ff @( posedge clk)
begin
  if (reset)
    q_r <= 0;
  else if ( enable )
    q_r <= q_n;
end
```

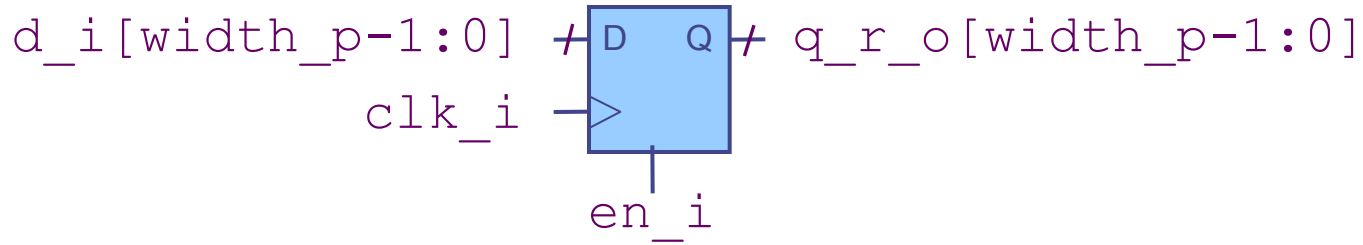
synchronous reset



Reset has priority over enable in this code.
Few reasons to prioritize enable over reset.

Register (i.e. a vector of flip-flops)

```
module register#(parameter width_p = 1)
(
  input  clk_i
 ,input  [width_p-1:0] d_i
 ,input  en_i
 ,output logic [width_p-1:0] q_r_o
);

  always_ff @( posedge clk_i )
  begin
    if (en_i)
      q_r_o <= d_i;
  end
endmodule
```



Implementing Wider Registers

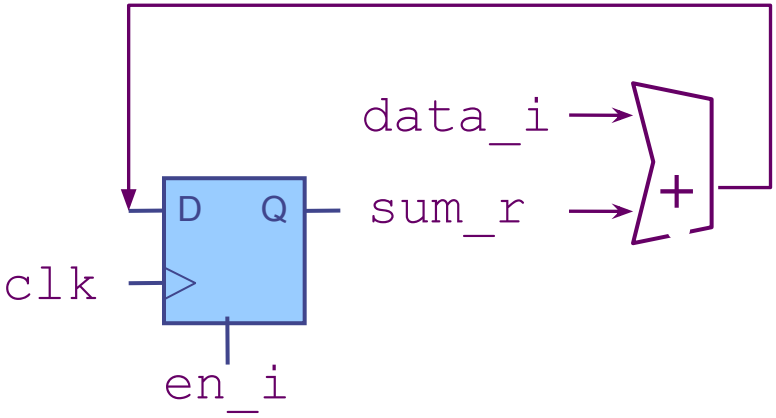
```
module register2
(input  clk,
,input  [1:0] d_i
,input  en_i
,output logic [1:0] q_r_o
);

 always_ff @(posedge clk)
  begin
    if (en_i)
     q_r_o <= d_i;
  end

endmodule
```

```
module register2
( input  clk
 ,input [1:0] d_i
 ,input  en_i
 ,output logic [1:0] q_r_o
);
 FF ff0 (.clk(clk)
         ,.d_i(d_i[0])
         ,.en_i(en_i)
         ,.q_r_o(q_r_o[0]));

 FF ff1 (.clk (clk)
         ,.d_i (d_i[1])
         ,.en_i(en_i)
         ,.q_r_o(q_r_o[1]));

endmodule
```

Do they behave the same?

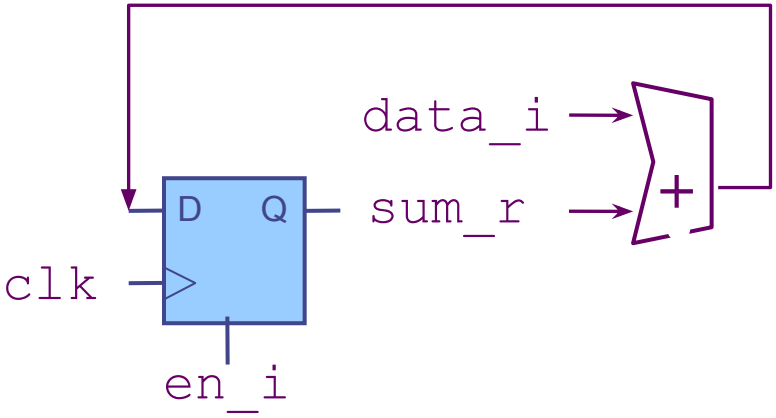
yes

Combined sequential/combinational idioms



Combined sequential/combinational idioms

```
module accum #(parameter width_p=1)
( input  clk
 ,input  data_i
 ,input  en_i
 ,output logic [width_p-1:0] sum_o
);

logic [width_p-1:0] sum_r;
assign sum_o = sum_r;

always_ff @(posedge clk)
  begin
    if (en_i)
     sum_r <= sum_r + data_i;
  end
```

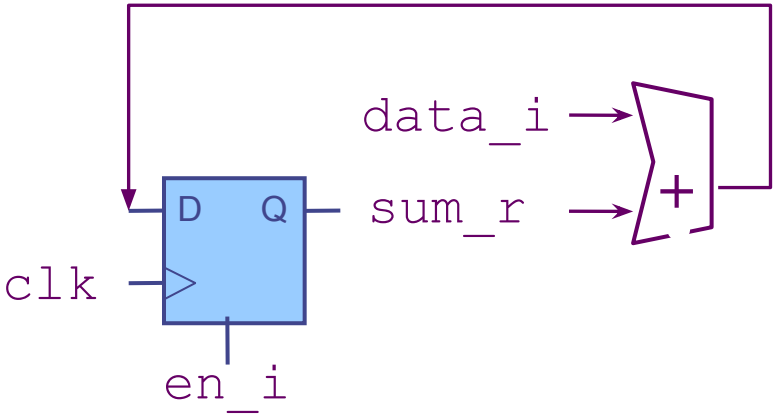


Combined sequential/combinational idioms

```
module accum #(parameter width_p=1)
( input  clk
 ,input  data_i
 ,input  en_i
 ,output logic [width_p-1:0] sum_o
);

logic [width_p-1:0] sum_r;
assign sum_o = sum_r;

always_ff @(posedge clk)
  begin
    if (en_i)
     sum_r <= sum_r + data_i;
  end
```

same

```
module accum #(parameter width_p=1)
( input  clk
 ,input  data_i
 ,input  en_i
 ,output logic [width_p-1:0] sum_o
);

logic [width_p-1:0] sum_r;
assign sum_o = sum_r;
wire [width_p-1:0] sum_n = sum_r + data_i;

always_ff @(posedge clk)
  begin
    if (en_i)
     sum_r <= sum_n;
  end
```



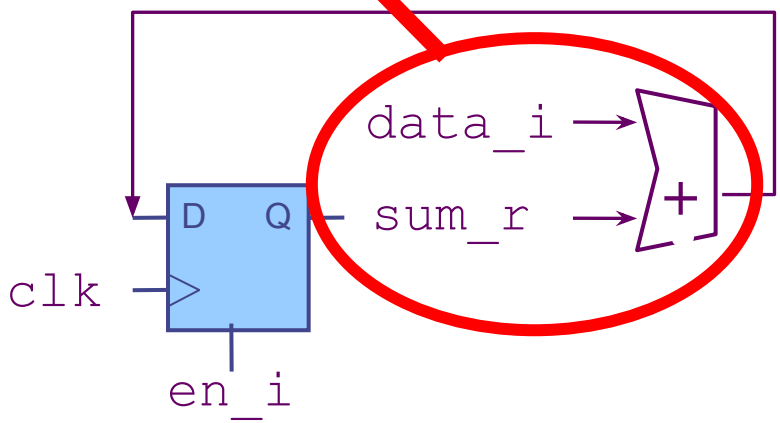
Combined sequential/combinational idioms

```
module accum #(parameter width_p=1)
( input  clk
 ,input  data_i
 ,input  en_i
 ,output logic [width_p-1:0] sum_o
);

logic [width_p-1:0] sum_r;
assign sum_o = sum_r;

always_ff @(posedge clk)
  begin
    if (en_i)
     sum_r <= sum_r + data_i;
  end
```

same

```
module accum #(parameter width_p=1)
( input  clk
 ,input  data_i
 ,input  en_i
 ,output logic [width_p-1:0] sum_o
);

logic [width_p-1:0] sum_r;
assign sum_o = sum_r;
wire sum_n = sum_r + data_i;

always_ff @(posedge clk)
  begin
    if (en_i)
     sum_r <= sum_n;
  end
```

combinational logic



Combined sequential/combinational idioms: Avoid nested conditional idioms with sequential logic – stick to simple enable and reset conditions.

```
module accum #(parameter width_p=1)
( input  clk
 ,input  data_i
 ,input  foo_i
 ,input  bar_i
 ,input  baz_i
 ,input  taz_i
 ,output logic [width_p-1:0] sum_o;
);

logic [width_p-1:0] A_r, B_r;
assign sum_o = A_r;

always_ff @(posedge clk)
  begin
    if (foo_i)
     A_r <= A_r + data_i;
    else
     if (bar_i)
       B_r <= B_r + 1;
     else
       if (baz_i)
         A_r <= B_r;
       else
        if (taz_i)
         B_r <= '0;
  end
```

Simulates in RTL,
Has Problems in
Gate Level Sim

Register files and SRAMs

SRAMs and register files should always be defined in a separate module.

We provide *bsg_mem_xxx* for the ones that translated well into hardware – use them!

```
module bsg_mem_1rw_sync
  #(parameter width_p=-1
    , parameter els_p=-1)
(input clk_i
, input reset_i
, input [width_p-1:0] data_i
, input [addr_width_lp-1:0] addr_i
, input v_i
, input w_i
, output logic [width_p-1:0] data_o
);
```

Also:

- bsg_mem_1r1w
- bsg_mem_2r1w
- bsg_mem_banked_crossbar
- bsg_ascii_to_rom.py

But in case you were wondering how you write these things in SystemVerilog: **Register array (or memory) idioms**

```
module mem_1r1w#(parameter width_p=-1
                 , parameter els_p=-1
                 , parameter lg_els_lp=$clog2(els_p))
( input  clk_i
 ,input                        w_v_i
 ,input [lg_els_p-1:0]         w_addr_i
 ,input [width_p-1:0]          w_data_i

 ,input [lg_els_p-1:0]         r_addr_i
 ,output logic [width_p-1:0] r_data_o;
);

logic [width_p-1:0] regarray [els_p-1:0];

always_ff @(posedge clk_i)
 if (w_v_i)
  regarray[w_addr_i] <= w_data_i;

assign r_data_o = regarray[r_addr_i];
```

Asynchronous Read,
Synchronous Write
1 read port,
1 write port

But don't write code like this.

Use BaseJump STL instead:

https://github.com/bespoke-silicon-group/basejump_stl/blob/master/bsg_mem/

See https://github.com/bespoke-silicon-group/basejump_stl/blob/master/bsg_mem/bsg_mem_multiport.v for a register array with a variable number of read and write ports!

Bit Manipulations

```
logic [15:0] x;
logic [31:0] x_sext;
logic [31:0] hi, lo;
logic [63:0] hilo;

// concatenation
assign hilo = { hi, lo};
assign { hi, lo } = { 32'b0, 32'b1 };

// duplicate bits (16 copies of x[15] + bits 15..0 of x)
assign  x_sext =  {{16 { x[15] }}, x[15:0]};

// select top_p bits starting at 0 (same as [top_p-1:0])
assign  foo = x[0+:top_p];
```

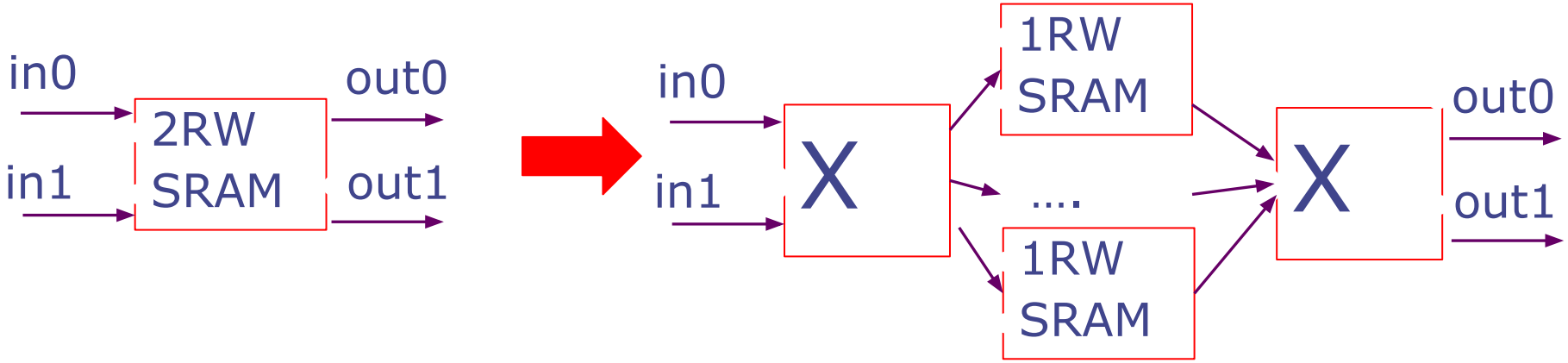
Memories: Register files (RAs) and SRAMs

Memories scale poorly in area, energy, and delay with # of ports:

Size = ~ width * els * R_ports * W_ports
Delay = ~ sqrt(size)
Energy = ~ sqrt(size)

Much of parallel architecture is about finding ways to avoid using large multiported SRAMS!

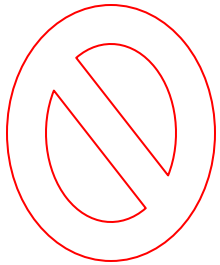
e.g. *banking*:



Beware of assignment shortcuts

```
logic [15:0] x;
assign x = y;
```

```
logic [15:0] x = y;
```



"initialization"
non-synthesizable

```
wire [15:0] x = y;
```

"continuous assignment"
synthesizable

"Unlike nets, a variable cannot have an implicit continuous assignment as part of its declaration. An assignment as part of the declaration of a variable is a variable initialization, not a continuous assignment."

`unique` and `priority` for `case` and `if`

`unique`: *exactly one* *branch or case item* ***must execute****; otherwise it is an error.*

`priority`: *choices* ***must*** *be* ***evaluated in order****, and that* ***one branch must execute****.*

Synopsys VCS: Does not generate X output, just says:

RT Warning: No condition matches in 'unique case' statement.
"system.v", line 20, for testbench.dut.cu, at time 100.

After synthesis, you will get a random value for the output! Sim Mismatch. So, using 1'bX as the default condition still has some purpose, since it shows up in the waveform viewer. On the other hand, this tells you when the issue happens.

```
module bar (input [1:0] foo);

logic a;

always_comb
begin
      a = 3;

      unique case (foo)
       0: a = 2;
       1: a = 1;
      endcase

End

endmodule
```

BAD: if foo != 0 and != 1;
then a is undefined after synthesis

```
module bar (input [1:0] foo);

logic a;

always_comb
begin
      unique case (foo)
       0: a = 2;
       1: a = 1;
       default: a = 3;
      endcase

End

endmodule
```

GOOD

```
module bar (input [1:0] foo);

logic a;

always_comb
begin
   a=3;

      unique case (foo)
       0: a = 2;
       1: a = 1;
       default:
      endcase

End

endmodule
```

GOOD

Note: Our SV Subset

- SV is a big language with many features not concerned with synthesizing hardware.
- The code you write for your hardware should only contain the language structures discussed in these slides.
- Anything else is not synthesizable, although it will simulate fine.
- You *MUST* follow the course coding standard.

EDA Playground

https://www.edaplayground.com/

- Cool website for trying out SystemVerilog!
  - Need an account to save designs, but to simply run something you don’t need an account
- You’ll need to choose a simulator
  - Circled in red on the next slide!
  - VCS requires registration (no big deal); you can also use Icarus Verilog.

https://www.edaplayground.com/

EDA Playground

EDA playground | Run | Save* | Follow the EDA Playground BLOG. | Playgrounds | Profile

Brought to you by DOULOS

Languages & Libraries

Testbench + Design

SystemVerilog/Verilog

UVM / OVM

None

Other Libraries

None
OVL 2.8.1
SVUnit 2.11

- Enable TL-Verilog
- Enable Easier UVM
- Enable VUnit

Tools & Simulators

Icarus Verilog 0.10.0 11/23/14

Compile & Run Options

-Wall -g2012

Run Options

- Open **EPWave** after run
- Download files after run

Examples

Community

Collaborate

Forum

Follow @edaplayground

testbench.sv

```
module test_adder;

  logic [7:0] a_li;
  logic [7:0] b_li;
  logic [7:0] sum_lo;
  logic       carry_lo;

  adder DUT
    (.a_i( a_li )
    ,.b_i( b_li )
    ,.sum_o( sum_lo )
    ,.carry_o( carry_lo )
    );

  initial begin

    // Test 1: 1 + 2 = 3
    a_li = 8'd1;
    b_li = 8'd2;
    #10
    if (sum_lo == 8'd3 && carry_lo == 1'b0)
      $display("TEST 1 -- PASS!");
    else
      $display("TEST 1 -- FAIL (sum=%d, carry=%d)", sum_lo, carry_lo);

    // Test 2: 1 + 2 = 3
    a_li = 8'd192;
    b_li = 8'd108;
    #10
    if (sum_lo == 8'd44 && carry_lo == 1'b1)
      $display("TEST 2 -- PASS!");
    else
      $display("TEST 2 -- FAIL (sum=%b, carry=%d)", sum_lo, carry_lo);

    $finish();

  end

endmodule
```

design.sv

SV/Verilog Design

```
module adder
  (input  [7:0] a_i
  ,input  [7:0] b_i
  ,output [7:0] sum_o
  ,output       carry_o
  );

  wire [8:0] sum_n = a_i + b_i;

  assign sum_o   = sum_n[7:0];
  assign carry_o = sum_n[8];

endmodule
```

Log | Share

```
[2019-01-07 13:44:42 EST] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
TEST 1 -- PASS!
TEST 2 -- PASS!
Done
```